

Guide pratique de l'assistant IA

TABLE DES MATIÈRES

1. Introduction

Ce que notre cerveau fait tout seul

Un collaborateur au CV hors norme

2. Travailler avec une IA

Être clair

Valider

Trois bonnes nouvelles

3. Le modèle et l'orchestrateur

Le modèle de langage

L'orchestrateur

Le secret de fabrication

4. C'est quoi la génération ?

Un modèle immobilier

Le LLM : même concept, autre échelle

D'où viennent les hallucinations

5. Le problème du poisson rouge

Pas de mémoire

La solution de l'orchestrateur

La fenêtre de contexte

Conclusion pratique

6. C'est quoi le contexte ?

Définition

Pourquoi "contexte" est bien choisi

Une limite fondamentale : du texte plat

Ce que l'orchestrateur vous offre

La mémoire

Le contexte dynamique

Skills et MCP

7. Le problème de l'attention

L'insight fondateur

Ce que ça coûte

Le problème de la dilution

Le "lost in the middle"

La fenêtre de contexte et la dilution

La self-attention et la polysémie

8. Conclusion

Index des mots-clés

Table des figures

Table des encadrés

1. Introduction ↑

Vous avez sans doute déjà utilisé un assistant IA — pour rédiger un email, poser une question, ou simplement tester ce dont il est capable. Vous avez peut-être été impressionné. Peut-être aussi déçu, ou surpris par une réponse à côté. Ce guide n'est pas un tutoriel. C'est une explication : comment ça marche vraiment, et comment en tirer le meilleur parti.

Ce que notre cerveau fait tout seul

Commençons par un aveu : nous sommes tous câblés pour prêter des intentions humaines à ce qui nous entoure. Un nuage qui "menace", une voiture qui "refuse de démarrer", un chat qui "boude". C'est ***anthropomorphisme*** — la tendance naturelle à projeter des caractéristiques humaines sur ce qui n'en a pas. Ce n'est pas un défaut de raisonnement : c'est une stratégie cognitive ancienne, efficace pour naviguer dans un monde social complexe.

Avec une IA, ce réflexe est particulièrement tenace. Elle vous répond en phrases construites, elle semble comprendre vos questions, elle s'adapte à votre ton. C'est le premier outil de l'histoire avec qui on *converse*. Difficile de ne pas lui prêter une forme d'**intention**.

Pourtant, une IA ne veut rien. Elle ne ressent rien. Elle n'a pas d'objectif propre, pas de bonne ou mauvaise journée, pas d'opinion sur vous. Ce point n'est pas une mise en garde morale — c'est une information utile pour travailler efficacement avec elle.

Un collaborateur au CV hors norme

Pour apprivoiser cette réalité, voici une image. Imaginez recruter un **collaborateur** avec un CV hors norme : la section *compétences* est immense — encyclopédie vivante, maîtrise de dizaines de langues, rédaction, analyse, code, droit, médecine, cuisine, poésie. Mais les sections *motivation* et *expérience professionnelle* sont vides. Il n'a pas de parcours, pas de raisons personnelles d'être là.

Et quelques particularités supplémentaires à bien noter pour la suite : il est distrait, il a une tendance marquée à vouloir faire plaisir coûte que coûte — parfois au détriment de l'exactitude — et il lui arrive d'inventer des choses. C'est ce qu'on appelle une **hallucination** : une réponse construite avec confiance, mais déconnectée de la réalité. Parfois flagrant, parfois beaucoup moins détectable.

Ce profil existe bel et bien. La bonne nouvelle : il est très gérable, si on sait comment l'aborder. C'est l'objet de ce guide.

2. Travailler avec une IA [↑]

Exploiter les qualités d'un collaborateur tout en limitant ses défauts — ça n'a rien de mystérieux. Deux principes suffisent.

Être clair

Une IA répond à ce qu'on lui demande, pas à ce qu'on voulait dire. Un **prompt** flou produit une réponse floue — ou pire, une réponse confiante qui part dans la mauvaise direction. Être précis dans ses questions et ses consignes, c'est la compétence de base.

Ça peut sembler contraignant. En pratique, c'est une bonne discipline générale. Avant d'envoyer un message, demandez-vous : *si on me disait ça, est-ce que je comprendrais sans ambiguïté ?* Cette question vaut pour les IA — et pour vos collègues.

Valider

Une IA ne sait pas qu'elle se trompe. Elle produit ce qui lui semble le plus probable, avec le même aplomb qu'elle soit juste ou complètement à côté. La **validation** des réponses n'est pas optionnelle — c'est une partie intégrante du travail. Plus l'enjeu est élevé, plus la vérification est nécessaire.

Trois bonnes nouvelles

Travailler avec rigueur ne veut pas dire travailler plus. Voici pourquoi.

Elle est très bienveillante. En anglais, on dirait *forgiving* — difficile à traduire exactement. Elle ne va pas se moquer de vos questions, ni se plaindre de vos reformulations. Vous pouvez tâtonner, recommencer, demander d'expliquer autrement, sans la moindre gêne.

Elle peut vous aider à progresser. Elle connaît très bien ses propres limites — elle ne se les applique juste pas spontanément. Vous pouvez lui demander de reformuler, d'expliquer son raisonnement, d'identifier ce qui manquait dans votre demande initiale. Si quelque chose ne va pas, construisez avec elle une **méthode de travail** meilleure. Une mise en garde cependant : ne la croyez pas quand elle dit que ça ne se reproduira plus. Ce n'est pas de la mauvaise volonté — c'est simplement qu'elle n'a pas de mémoire d'une session à l'autre. On reviendra là-dessus.

La documentation suit naturellement. Travailler en documentant au fur et à mesure est normalement pénible. Avec une IA, si on prend ce pli dès le début, c'est fluide : elle rédige, reformule, structure. Le projet et sa méthode avancent ensemble.

3. Le modèle et l'orchestrateur [↑]

Un assistant IA n'est pas un seul composant monolithique. Sous le capot, il y en a au moins deux — et comprendre leur rôle respectif change la façon dont on l'utilise.

Le modèle de langage

Le premier composant est le **modèle de langage** — souvent désigné par l'acronyme **LLM** (*Large Language Model*). Son rôle est précis : compléter un texte inachevé.

Prenons un exemple simple. La phrase *"La capitale de la France est"* reste suspendue. Sa complétion la plus probable est *"Paris"*. Le consensus est fort — c'est ce que dit la grande majorité des textes sur lesquels le modèle a été entraîné. Vichy reste une réponse techniquement plausible, mais très minoritaire.

Maintenant : *"La meilleure équipe de foot en France est"*. C'est plus intéressant. Plusieurs complétions sont plausibles, le consensus moins net.

Reformulons encore : *"De l'avis des principaux commentateurs sportifs, les trois meilleures équipes de foot en France sont"*. La **complétion** devient plus riche, plus nuancée. C'est déjà une introduction au rôle du **prompt** : la façon de formuler oriente le résultat.

L'orchestrateur

Le second composant est l'**orchestrateur**. C'est lui qui fait l'interface avec vous. Il reçoit votre message, le prépare pour le modèle, envoie le tout, récupère la réponse, et vous l'affiche.

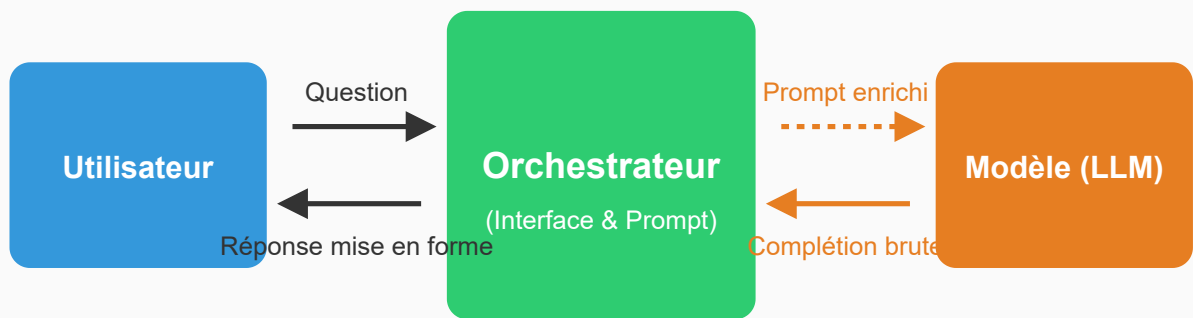


Figure 1 — Diagramme du flux entre les deux composants : humain → prompt → orchestrateur → prompt étendu → modèle → complétion → orchestrateur → réponse → humain. Flux aller (prompt) et retour (réponse) distincts entre chaque composant. Trois nœuds : humain (gris, neutre), orchestrateur (teal), modèle/LLM (violet). Style épuré, flèches directionnelles, labels courts.

Le secret de fabrication

Personne ne veut parler directement au modèle brut — en lui soumettant des débuts de phrase à compléter. C'est là qu'intervient l'orchestrateur : avant votre question, il glisse un ensemble d'instructions que vous ne voyez pas. C'est ce qu'on appelle le **prompt système**.

Ça ressemble à quelque chose comme :

Tu es un assistant IA qui répond à une question posée par un utilisateur. Tu dois donner des informations claires et précises, avec une explication courte, voire plusieurs possibilités s'il y a des ambiguïtés. Tu poses des questions de clarification si nécessaire. Tu peux ajouter quelques informations complémentaires qui restent dans le thème, et tu relances la conversation avec une question ouverte pour engager des demandes d'informations complémentaires. [...]

Avec ce prompt système, la complétion de *"La capitale de la France est"* pourrait donner :

La capitale de la France est Paris, sur la Seine. C'est aussi la plus grande ville du pays avec 2 millions d'habitants. Y a-t-il d'autres pays dont vous voudriez connaître

Cet exemple est imaginaire — il illustre simplement l'importance du prompt système : ton, format, gestion des relances, comportement face aux ambiguïtés. C'est un secret de fabrication complexe, finement réglé, propre à chaque assistant.

Votre propre formulation compte autant. Mais si on comprend le principe, c'est d'abord du bon sens : être clair, fournir du contexte. N'écoutez pas trop les pseudo-gourous qui vendent des prompts soi-disant tunés — *"Tu es un professeur de danse spécialisé dans les claquettes..."* Ce genre de formule peut même être contre-productif, comme on le verra plus loin.

4. C'est quoi la génération ? [↑]

Il y a quelque chose de presque magique dans ce que fait un LLM. On comprend que ses connaissances viennent de textes humains — mais nulle part dans la littérature humaine il n'existe un texte aussi spécifique que le verbiage d'un prompt système sur mesure. Comment le modèle peut-il produire quelque chose d'utile à partir de ça ?

La réponse tient en un mot : interpolation. Pour l'expliquer, partons d'un exemple volontairement simple.

Un modèle immobilier

Je veux estimer la valeur d'un appartement. Je dispose d'une liste de transactions immobilières avec les surfaces et les prix.

Approche classique : calculer le prix moyen au m², puis multiplier par la surface de mon appartement.

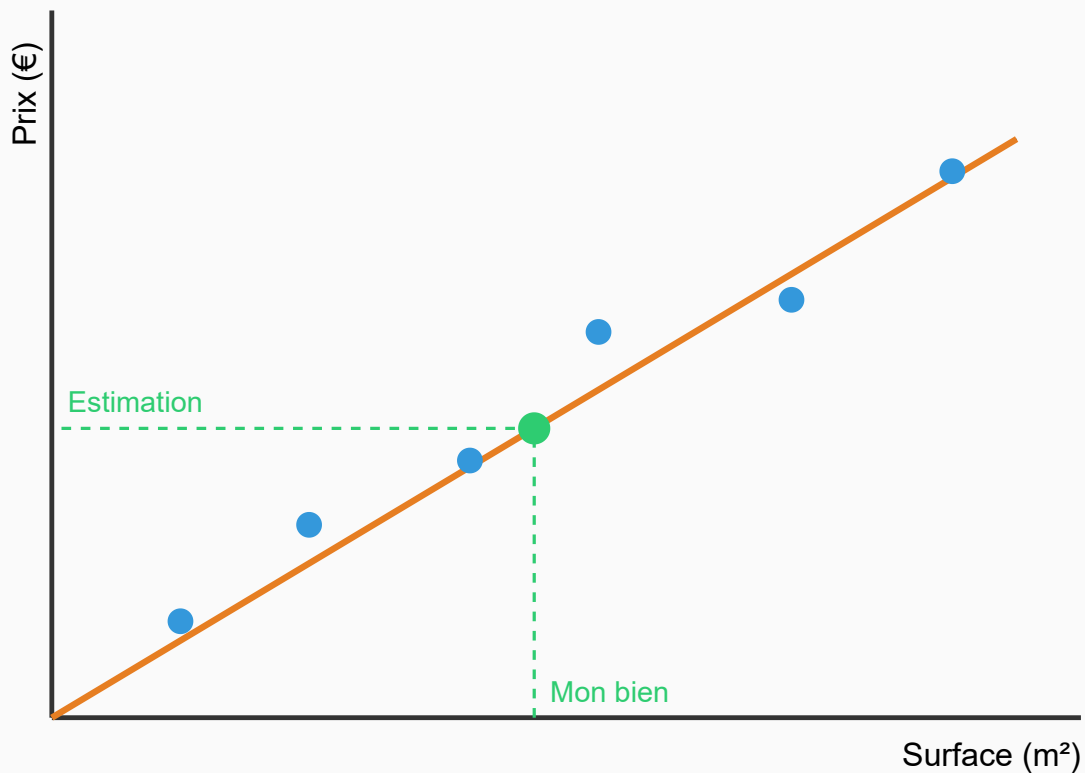


Figure 2 — Graphique axes surface (x) / prix (y). Points représentant les transactions. Droite de régression représentant le prix moyen au m². Ligne verticale rouge partant de la surface de l'appartement cible, ligne horizontale arrivant à son prix estimé. Style pédagogique, épuré.

Sans le savoir, je viens de construire un modèle. Un modèle, c'est trois choses :

- une **procédure de calcul** : multiplier par le prix au m²
- des **paramètres** qui règlent ce calcul : ici un seul, le prix moyen au m²
- une **méthode d'apprentissage** pour trouver les paramètres à partir de données connues — ici les transactions immobilières, et la méthode le calcul de la moyenne

Même si la surface de mon appartement ne figure dans aucune transaction historique, ou si plusieurs transactions ont la même surface avec des prix différents, j'obtiens une estimation raisonnable. C'est ça, la magie de l'interpolation : généraliser à partir d'exemples, sans avoir besoin d'un exemple pour chaque cas.

Le LLM : même concept, autre échelle

Un LLM fonctionne sur le même principe, à une échelle radicalement différente :

- **Entrées/sorties** : des débuts de texte et leur complétion
- **Données d'entraînement** : une immense base de textes, chacun découpé en couples (début, complétion)
- **Procédure de calcul** : c'est la **génération** — complexe, mais rapide à exécuter une fois le modèle construit (les fameux GPU y contribuent)
- **Paramètres** : plusieurs milliards, parfois dizaines de milliards
- **Apprentissage** : extrêmement coûteux en calcul, qui ne se refait pas tous les jours

D'où viennent les hallucinations

Notre modèle immobilier peut donner des évaluations irréalistes. Pourquoi ? Deux raisons : la qualité des données et la puissance du modèle.

La qualité des données. *Garbage in, garbage out.* Un exemple typique : des valeurs aberrantes — un bien bradé, un vendu très au-dessus du marché — qui faussent le calcul des paramètres.

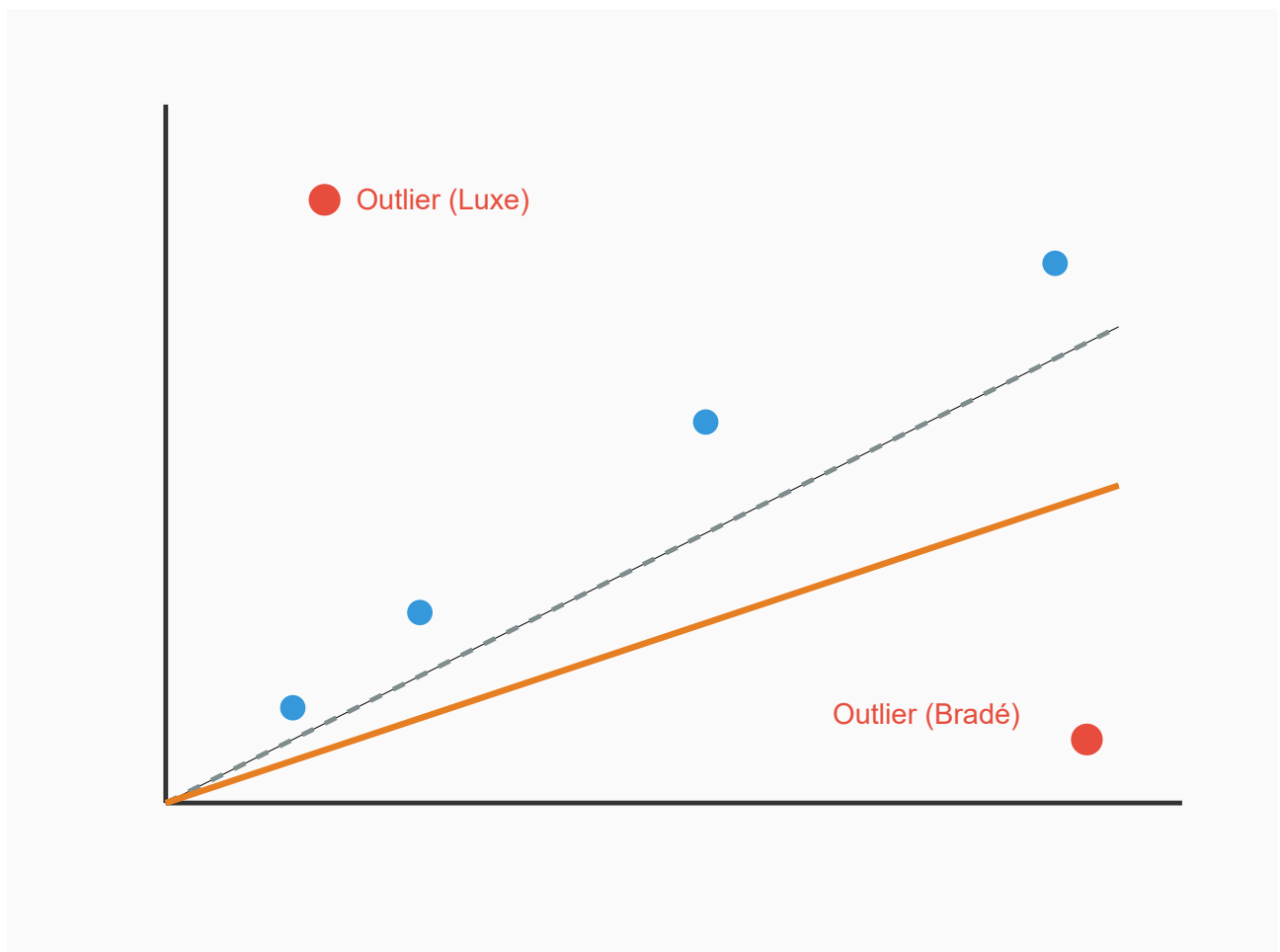


Figure 3 — Mêmes données que immo-regression, avec quelques points aberrants bien exagérés (un bien bradé, un vendu très au-dessus du marché). Montrer comment ils dévient la droite de régression.

Pour les LLM, c'est le même principe : gros travail de préparation des données, filtrage, nettoyage. Wikipedia, c'est bien. Les forums complotistes, beaucoup moins.

La puissance du modèle. Il y a peut-être plus que juste la surface : quartier, âge de l'immeuble, étage... Et même avec une seule variable, la puissance du modèle joue. Les petites surfaces sont souvent plus chères au m^2 — notre droite ne capture pas ça. On peut enrichir le modèle : deux pentes, une pour les petites surfaces, une pour les grandes.

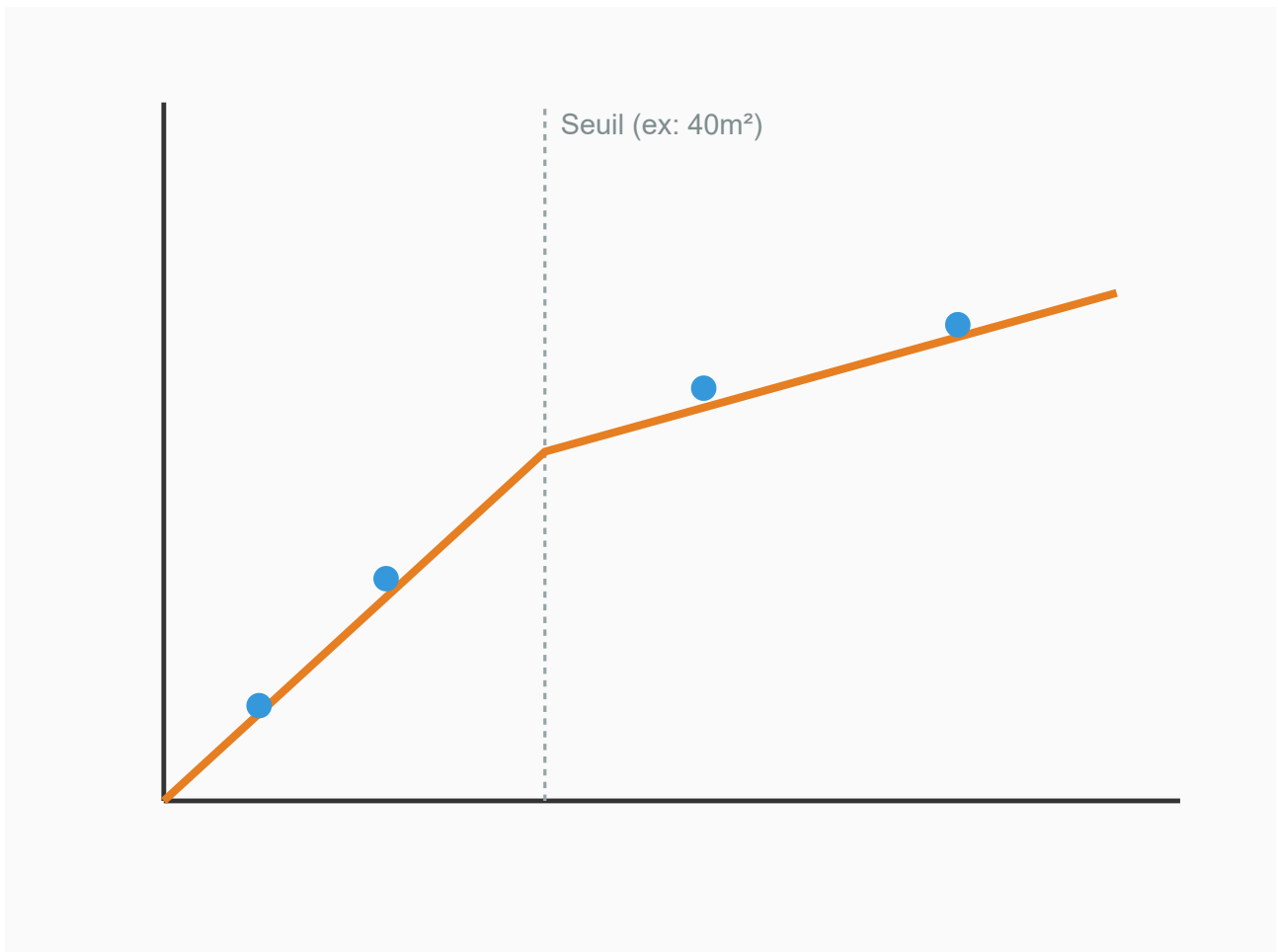


Figure 4 — Modèle à deux pentes : une pour les petites surfaces (prix/ m^2 élevé), une pour les grandes (prix/ m^2 plus faible). Illustre l'enrichissement du modèle par rapport à immo-regression.

Quand un modèle n'est pas assez puissant pour produire des résultats satisfaisants, on dit qu'il fait de l'**underfitting**. Vous aurez beau l'entraîner davantage, il ne progressera pas — il lui manque la structure pour répondre à la question.

À l'inverse, un modèle très complexe avec trop de paramètres par rapport aux données disponibles peut "apprendre par cœur" les exemples sans généraliser. Sa courbe passe par tous les points connus, mais part dans des directions improbables entre eux. C'est l'**overfitting**.

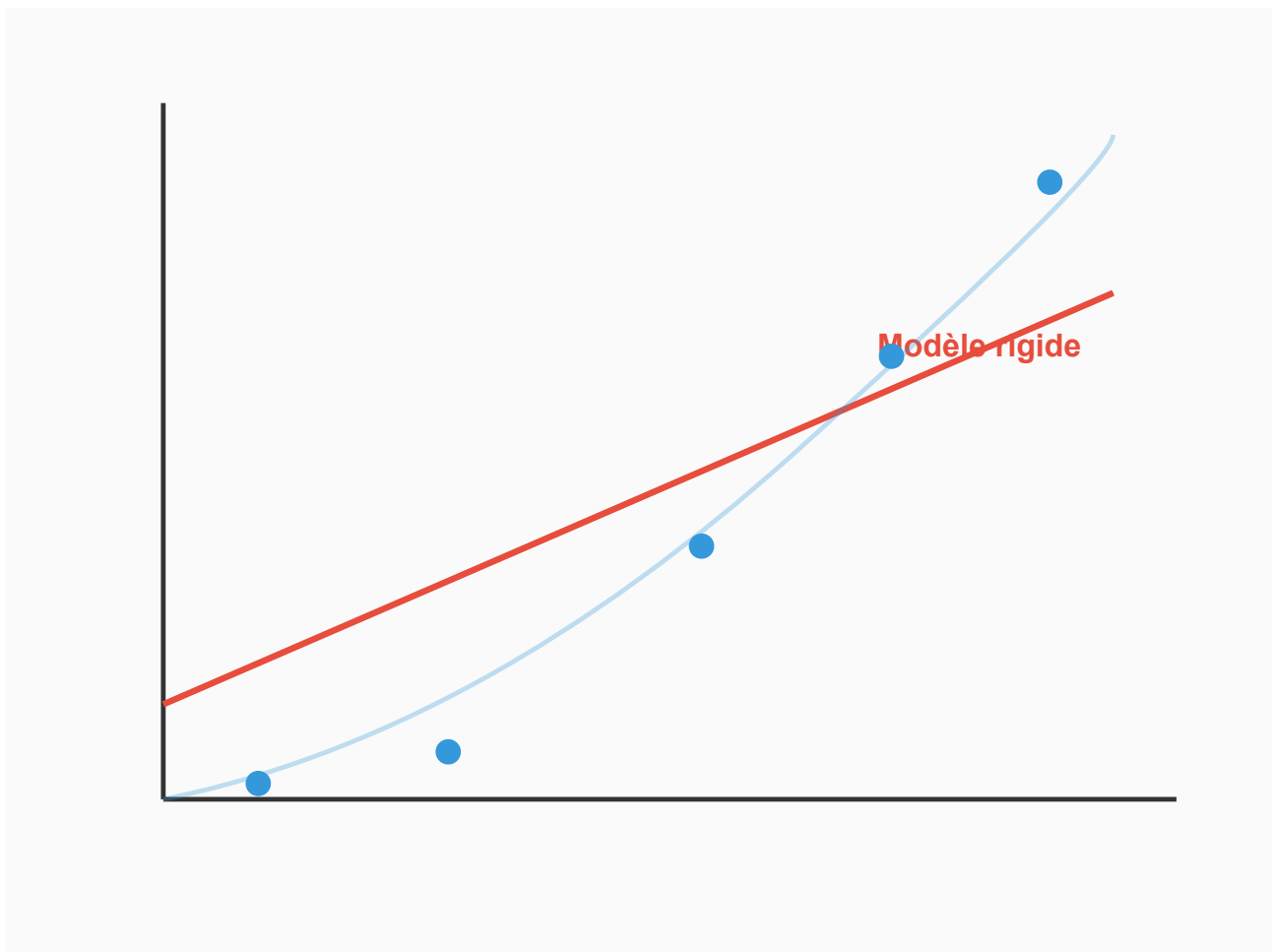


Figure 5 — Données avec une structure visible (courbe), modèle trop simple (droite) qui ne capture pas la tendance. Le modèle "n'est pas assez intelligent".

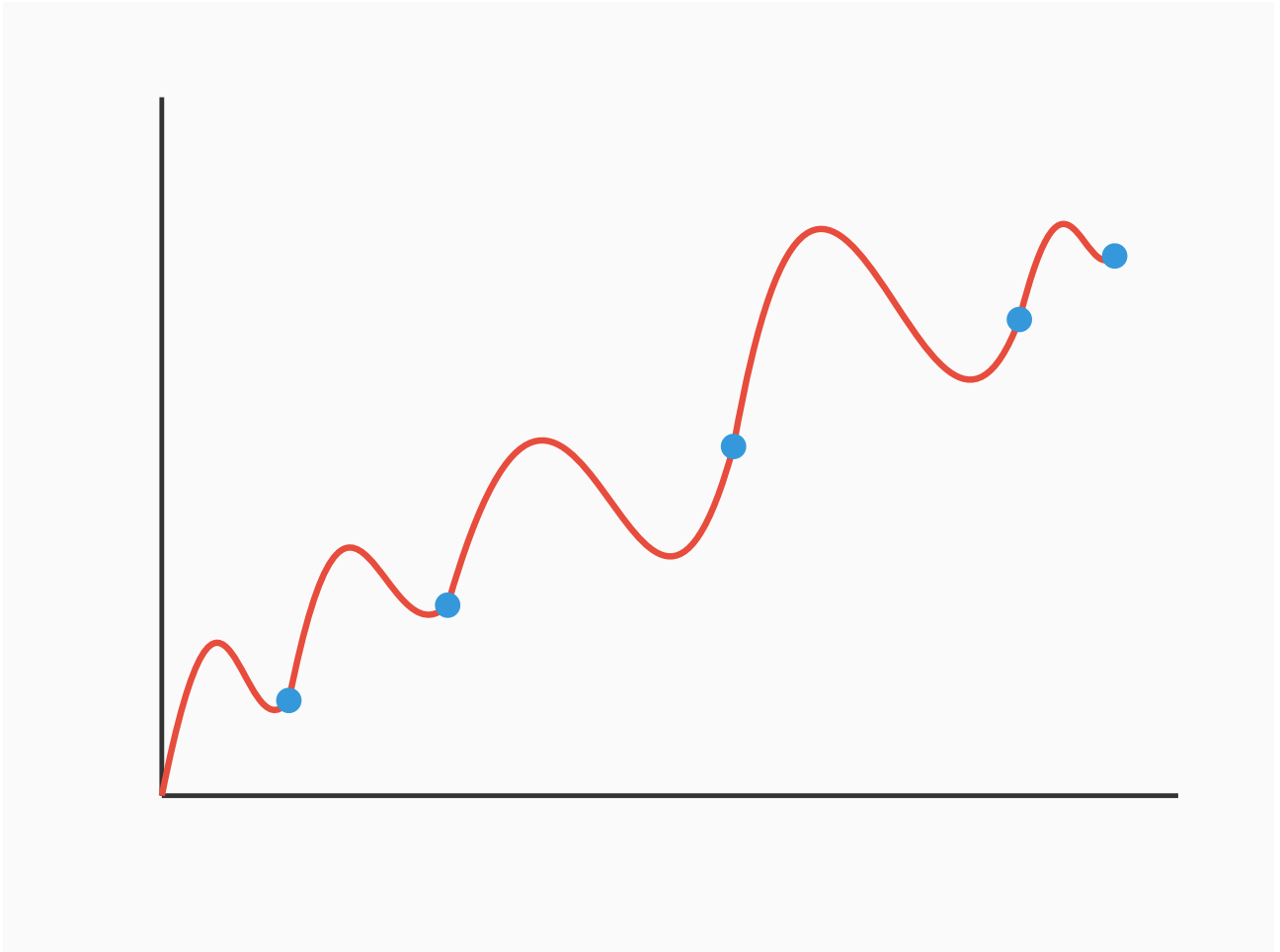


Figure 6 — Même données, modèle trop complexe dont la courbe passe par tous les points mais part dans des directions improbables entre les données. Le modèle "apprend par cœur" au lieu de généraliser.

Techniquement, on parle d'**interpolation** quand les questions tombent *entre* les données d'entraînement, et d'**extrapolation** quand elles tombent *en dehors*. Dans les deux cas, le modèle fait de son mieux — mais extrapoler l'amène encore plus facilement dans la science-fiction.

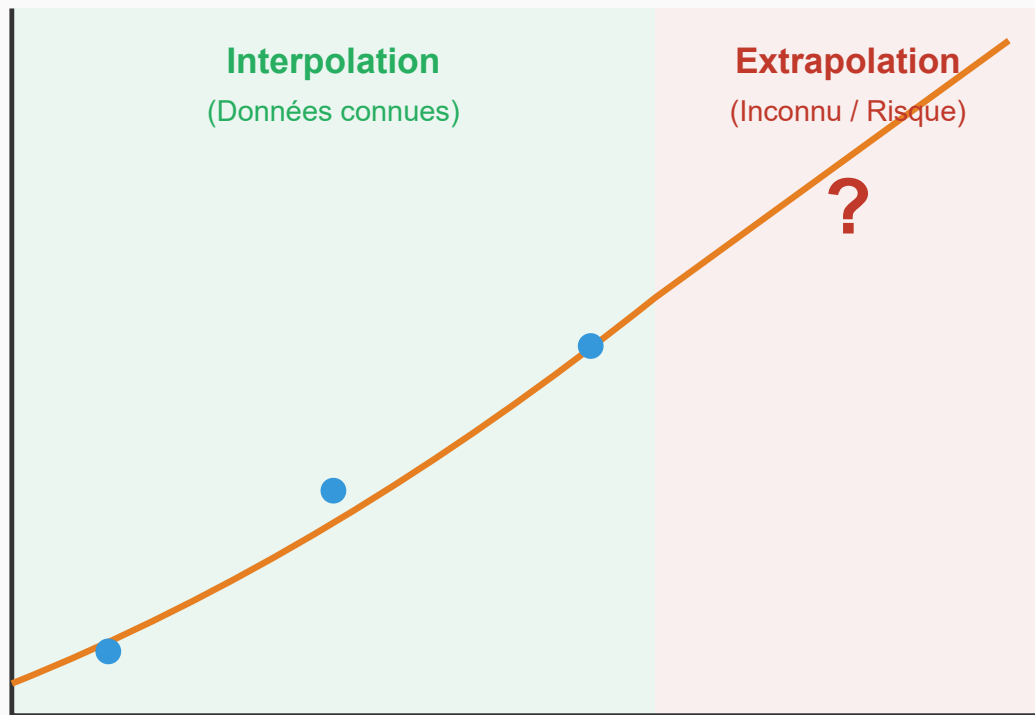


Figure 7 — Distinction visuelle : zone entre les données d'entraînement (interpolation, en vert) et zones en dehors (extrapolation, en rouge). Montrer qu'extrapoler amplifie les erreurs du modèle.

5. Le problème du poisson rouge ↑

Le poisson rouge a la réputation d'oublier tout toutes les trente secondes. C'est inexact — mais c'est une bonne métaphore pour décrire un aspect fondamental des LLM.

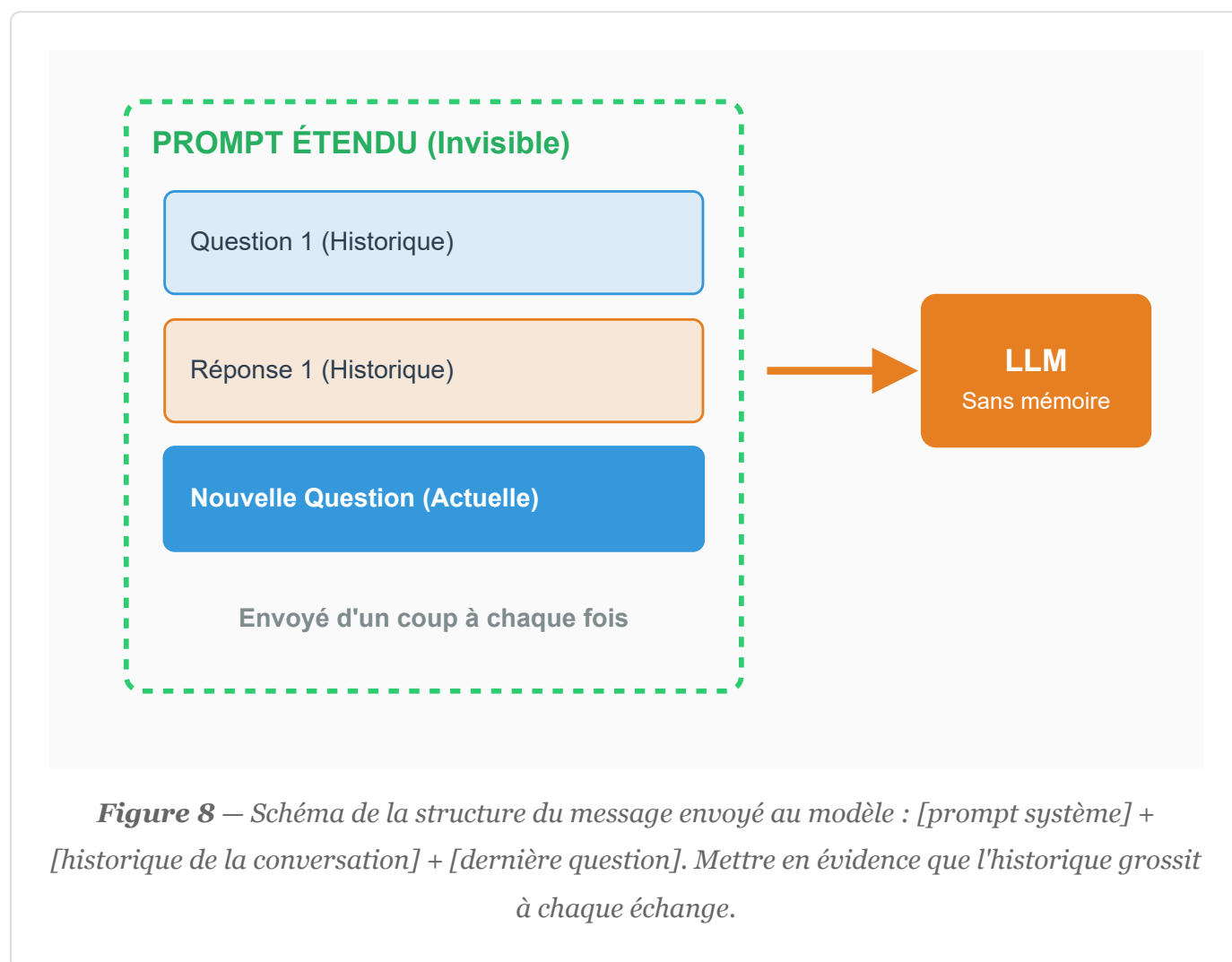
Pas de mémoire

Entre deux générations, le modèle repart de zéro. Aucune mémoire, aucun état — on dit qu'il est **stateless (sans état)**. Chaque génération est entièrement indépendante de la précédente.

C'est un choix d'architecture, pas un oubli de conception. Mais ça pose un problème évident : comment tenir une conversation si on oublie tout à chaque réponse ?

La solution de l'orchestrateur

L'*orchestrateur* s'en charge. À chaque échange, il reconstruit un message complet incluant tout l'**historique** de la conversation, et l'envoie au modèle. Le modèle ne se souvient de rien — mais il reçoit tout.



Ce message complet s'appelle tantôt "prompt", tantôt "**contexte**" — parce qu'il contient à la fois votre question et les éléments de contexte nécessaires pour y répondre. C'est la même chose, vue sous deux angles différents. Le contexte, c'est de la mémoire artificielle.

(Parenthèse involontairement illustrative : ce paragraphe vient d'utiliser le mot "contexte" en lui donnant deux sens légèrement différents — exactement le genre d'ambiguïté que les chapitres suivants vont explorer.)

La fenêtre de contexte

Ce message a une taille limite : la **fenêtre de contexte**. Elle dépend du modèle, et elle est très grande — mais l'historique croît à chaque échange. Quand on approche de la limite, l'orchestrateur peut compresser les échanges anciens pour faire de la place.

Tout ce texte a un coût : c'est ce qu'on appelle les **tokens** — l'unité de mesure des LLM. Roughly un mot ou un fragment de mot.

Un contexte très long peut aussi créer de la confusion pour le modèle. On verra pourquoi dans le chapitre sur l'attention.

Conclusion pratique

Des conversations courtes, centrées sur un seul sujet. Si le sujet change, commencez une nouvelle session.

(Si toutes les réunions étaient organisées comme ça...)

6. C'est quoi le contexte ? ↑

On a déjà croisé ce mot plusieurs fois. Il est temps de le définir proprement.

Définition

Le **contexte** — ou **prompt étendu** — c'est tout ce qu'on envoie au modèle, au-delà de votre question immédiate. On en a déjà vu deux exemples : le prompt système, ajouté par l'orchestrateur, et l'historique de la session. Le contexte a plusieurs rôles : contrôler le style de la réponse, orienter les priorités, et donner de la mémoire artificielle au modèle.

Pourquoi "contexte" est bien choisi

Le langage humain est par essence ambigu. "Tu peux me passer le sel ?" n'est pas une question sur vos capacités physiques. Comprendre ce que veulent dire les mots dépend des

autres mots autour d'eux — c'est précisément ce mécanisme qui a débloqué la technologie des LLM. On y reviendra au chapitre suivant.

Une limite fondamentale : du texte plat

Le LLM ne se regarde pas travailler. Il reçoit un grand texte à compléter — et c'est tout. Il ne peut pas hiérarchiser mécaniquement les éléments du contexte : on lui dit ce qui est important, et on espère qu'il comprend.

Point crucial : le contexte est toujours **déclaratif, jamais impératif**. Quand vous écrivez *"réponds en trois points"*, comprenez : *"la probabilité que la réponse contienne trois points est plus forte"*. Ce n'est pas un ordre exécuté — c'est une intention formulée. Certaines formulations sont plus efficaces que d'autres, et on verra comment en tirer parti.

Ce que l'orchestrateur vous offre

L'orchestrateur propose des outils pour enrichir le contexte de façon statique. Vous pouvez enregistrer des **instructions utilisateur** personnelles qui seront automatiquement insérées dans chaque prompt étendu.

Anatomie du Prompt Étendu (Contexte Global)

1. Prompt Système

Rôle, consignes de style, limites

2. Instructions Utilisateur

Préférences globales ("sois neutre")

3. Documents & Connaissances

Fichiers joints, base Projet (RAG)

4. Mémoire à long terme

Notes capitalisées au fil des sessions

5. Historique de la session

Échanges de messages précédents (mémoire artificielle à court terme)

6. Question de l'utilisateur (Le Prompt Actuel)

Figure 9 — Schéma de la structure complète du prompt étendu avec toutes les couches :
[prompt système] → [instructions utilisateur] → [historique de la session] → [dernier prompt].
Variante enrichie de la figure prompt-etendu-historique.

Certains orchestrateurs vont plus loin. Claude propose par exemple la notion de **projet** : un contexte partagé par un groupe de sessions — des instructions, des documents de référence, une mémoire commune.

Un projet peut inclure des fichiers de référence — documentations, notes, exemples — injectés dans chaque prompt. Utile quand leur contenu est fondamental. Mais attention : ce sont des tokens supplémentaires, et un contexte qui grossit trop risque de diluer le focus. On retrouve ici le conseil du chapitre précédent : sessions courtes et centrées.

La mémoire

L'orchestrateur peut gérer une **mémoire** — globale ou par projet. Elle est injectée dans le prompt, clairement balisée. Le prompt système indique au modèle qu'une mémoire est disponible, et l'invite à proposer des mises à jour si la session apporte des éléments pertinents. En résumé : le modèle peut prendre des notes pour les prochaines sessions.

Avantage : une mémoire qui se construit dans le temps. Inconvénient : on ne maîtrise pas toujours bien ce qui s'y accumule — un peu comme les cookies publicitaires.

Pour un usage professionnel : restreindre la mémoire au niveau projet, voire s'en passer. La réserver aux usages plus personnels.

Le contexte dynamique

Jusqu'ici, le contexte était statique : défini à l'avance, injecté mécaniquement. Il existe une approche plus subtile : laisser le modèle lui-même choisir le contexte dont il a besoin.

Concrètement, au lieu d'un seul tour de génération par question, on en fait plusieurs. Une génération peut inclure des demandes de contexte supplémentaire, que l'orchestrateur va chercher avant de relancer la génération.

Exemple : *"Fait-il beau à Nice ?"* Les données d'entraînement du modèle sont passées — la météo du jour n'y figure pas. Le modèle sait pourtant beaucoup de choses utiles : que cette

question relève de la météo, que la météo s'applique à un lieu, que Nice est une ville, et que la façon la plus pratique d'obtenir la météo en temps réel est de faire une recherche web. Il sait probablement quel service appeler.

Sa réponse va donc contenir une demande : effectuer cette recherche et lui communiquer le résultat. L'orchestrateur exécute, récupère l'information, reconstruit le prompt avec le résultat, et relance la génération. Le modèle produit alors une réponse utile.

(Tout l'anthropomorphisme de cet exemple est assumé pour la lisibilité — il n'y a bien sûr ni intention ni mémoire d'une demande à l'autre.)

Voir encadré *voir encadré undefined* pour la mécanique sous-jacente.

Skills et MCP

La recherche web n'est qu'un exemple. Le **contexte dynamique** repose sur un concept plus général : des **outils** que le modèle peut solliciter.

Un **skill** est un ensemble d'instructions décrivant une compétence à activer à la volée. Exemple : comment lire un fichier Excel. Le modèle reçoit le contenu brut du fichier et des instructions décrivant sa structure — il peut alors l'exploiter correctement.

Le concept ultime est le **MCP** — *Model Context Protocol*, défini par Anthropic. Un MCP est une description fournie à l'orchestrateur pour accéder à un service externe. Il expose une liste de commandes disponibles et des instructions en langage naturel décrivant quand et comment les utiliser.

L'industrie a réagi avec enthousiasme : on trouve maintenant des centaines de MCP permettant, en langage naturel, de lire et envoyer des emails, d'accéder à des fichiers et au cloud, d'interroger des bases de données, de créer des factures dans un ERP, de générer des modèles 3D...

7. Le problème de l'attention ↑

Ce chapitre revient sur le mécanisme qui a tout changé — et explique pourquoi les contextes longs peuvent nuire à la qualité des réponses.

L'insight fondateur

Le langage est ambigu par nature. Ce n'est pas un défaut — c'est sa caractéristique fondamentale. Le sens d'un mot dépend des mots qui l'entourent.

ENCADRÉ 1

Prenez le mot *tour*. Tour Eiffel, Tour de France, tour de potier, pièce aux échecs, tour de chant, tour de vis, ordre dans une file, donjon médiéval... Et "*il grimpe dans les tours*" : régime moteur ou quelqu'un qui s'énervé ?

C'est précisément ce défi — résoudre l'ambiguïté du langage — qui a conduit à la percée technologique des LLM : l'**attention**. Chaque mot "regarde" tous les autres et calcule à quel point ils sont pertinents pour lui. Ce mécanisme permet de pondérer l'ensemble du contexte simultanément, et non mot à mot.

Ce que ça coûte

Ce mécanisme est puissant mais exigeant : son coût de calcul croît très vite avec la longueur du contexte. Plus le contexte est long, plus le calcul est lourd — et plus la qualité peut se dégrader.

Le problème de la dilution

Un contexte long ne signifie pas une meilleure compréhension. L'**attention** doit se répartir sur l'ensemble du texte. Si ce texte contient beaucoup d'éléments peu pertinents, la concentration se dilue sur des informations inutiles.

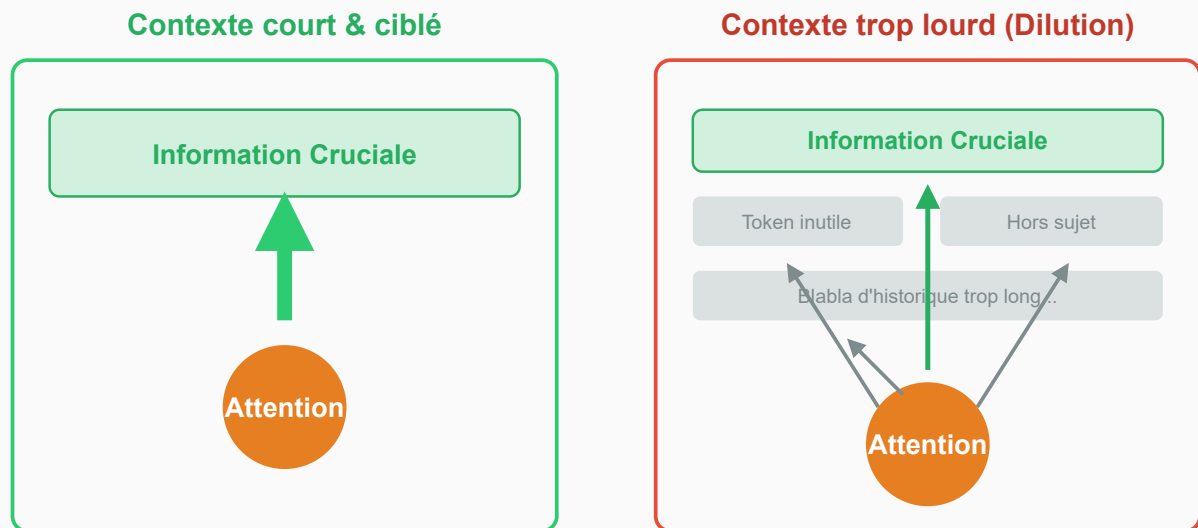


Figure 10 — Illustration du mécanisme de dilution de l'attention : un contexte court où l'attention est concentrée sur les éléments pertinents, vs un contexte long où elle se disperse sur de nombreux éléments peu pertinents.

C'est contre-intuitif : on a l'impression qu'en donnant plus d'informations, on aide. En réalité, on peut nuire.

Analogie : un jury qui doit trancher un litige. Lui soumettre 10 documents clairs est plus efficace que 200 pièces dont 190 sont hors sujet. La pertinence prime sur le volume.

Le "lost in the middle"

Dans un contexte très long, les informations placées au milieu sont statistiquement moins bien traitées que celles en début ou en fin de contexte. C'est ce qu'on appelle le phénomène de **lost in the middle**.

Implication pratique : si vous avez une information critique, ne la noyez pas au milieu d'un long document.

La fenêtre de contexte et la dilution

Ces deux contraintes — taille maximale et dégradation par dilution — convergent vers les mêmes conseils pratiques :

- Des prompts courts et ciblés
- Les informations importantes en tête ou en queue de contexte
- Ne pas injecter des documents entiers si seules quelques sections sont pertinentes
- Reconnaître les signes de dégradation : réponses qui ignorent des instructions données, incohérences avec des éléments mentionnés plus tôt dans la session

La *self-attention* et la *polysémie*

Le mécanisme d'attention résout la polysémie — mais il le fait dans les deux sens. Tout comme le contexte aide à désambiguïser un mot, un contexte trop chargé ou mal structuré peut introduire de nouvelles ambiguïtés. Un document qui traite de plusieurs sujets à la fois peut amener le modèle à mélanger des fils pourtant distincts.

C'est une raison supplémentaire pour la règle d'or : une session, un sujet.

8. Conclusion [↑]

Ce guide a parcouru le fonctionnement d'un assistant IA, de l'architecture de base jusqu'aux mécanismes qui expliquent ses forces et ses limites.

Quelques points à garder en tête.

Ce n'est pas de la magie — et c'est mieux comme ça. Comprendre que la génération repose sur l'interpolation, que la mémoire est artificielle, que le contexte est déclaratif — ce n'est pas désenchantant. Au contraire : ça permet de travailler de façon plus efficace, et d'éviter les déceptions inutiles.

Les limites sont structurelles, pas accidentelles. Les hallucinations, la sensibilité au contexte, l'absence de mémoire native — ce ne sont pas des bugs à corriger. Ce sont des caractéristiques architecturales. Travailler avec une IA, c'est travailler *avec* ces caractéristiques, pas contre elles.

Quelques réflexes suffisent. Être clair dans les demandes. Valider les résultats. Garder des sessions courtes et centrées. Placer les informations importantes en début de contexte.

Documenter au fil de l'eau. Ces réflexes ne demandent pas de compétences techniques — ils demandent de la méthode.

C'est un outil qui évolue vite. Les modèles s'améliorent, les orchestrateurs s'enrichissent, les usages se diversifient. Ce qui est vrai aujourd'hui sur les fenêtres de contexte ou les hallucinations le sera peut-être moins demain. Mais les principes de base — comprendre ce qu'on utilise, adapter sa façon de travailler, garder un œil critique — restent valables quelle que soit la génération technologique.

Bonne collaboration.

Index des mots-clés ↑

TERME	DÉFINI AU
<i>anthropomorphisme</i>	Chapitre 1 — Introduction ↑
<i>attention</i>	Chapitre 7 — Le problème de l'attention ↑
<i>bienveillance</i>	Chapitre 2 — Travailler avec une IA ↑
<i>collaborateur</i>	Chapitre 1 — Introduction ↑
<i>complétion</i>	Chapitre 3 — Le modèle et l'orchestrateur ↑
<i>contexte</i>	Chapitre 6 — C'est quoi le contexte ? ↑
<i>déclaratif</i>	Chapitre 6 — C'est quoi le contexte ? ↑
<i>dilution</i>	Chapitre 7 — Le problème de l'attention ↑
<i>données</i>	Chapitre 4 — C'est quoi la génération ? ↑
<i>extrapolation</i>	Chapitre 4 — C'est quoi la génération ? ↑
<i>fenêtre</i>	Chapitre 7 — Le problème de l'attention ↑
<i>génération</i>	Chapitre 4 — C'est quoi la génération ? ↑
<i>hallucination</i>	Chapitre 4 — C'est quoi la génération ? ↑
<i>historique</i>	Chapitre 5 — Le problème du poisson rouge ↑

TERME	DÉFINI AU
<i>instructions</i>	Chapitre 6 — C'est quoi le contexte ? ↑
<i>intention</i>	Chapitre 1 — Introduction ↑
<i>interpolation</i>	Chapitre 4 — C'est quoi la génération ? ↑
<i>LLM</i>	Chapitre 3 — Le modèle et l'orchestrateur ↑
<i>lost</i>	Chapitre 7 — Le problème de l'attention ↑
<i>MCP</i>	Chapitre 6 — C'est quoi le contexte ? ↑
<i>mémoire</i>	Chapitre 6 — C'est quoi le contexte ? ↑
<i>méthode</i>	Chapitre 2 — Travailler avec une IA ↑
<i>modèle</i>	Chapitre 3 — Le modèle et l'orchestrateur ↑
<i>orchestrateur</i>	Chapitre 3 — Le modèle et l'orchestrateur ↑
<i>outil</i>	Chapitre 6 — C'est quoi le contexte ? ↑
<i>overfitting</i>	Chapitre 4 — C'est quoi la génération ? ↑
<i>polysémie</i>	Chapitre 7 — Le problème de l'attention ↑
<i>projet</i>	Chapitre 6 — C'est quoi le contexte ? ↑
<i>prompt</i>	Chapitre 6 — C'est quoi le contexte ? ↑
<i>sans</i>	Chapitre 5 — Le problème du poisson rouge ↑
<i>self-attention</i>	Chapitre 7 — Le problème de l'attention ↑
<i>skill</i>	Chapitre 6 — C'est quoi le contexte ? ↑
<i>stateless</i>	Chapitre 5 — Le problème du poisson rouge ↑
<i>token</i>	Chapitre 5 — Le problème du poisson rouge ↑
<i>underfitting</i>	Chapitre 4 — C'est quoi la génération ? ↑
<i>validation</i>	Chapitre 2 — Travailler avec une IA ↑

Table des figures [↑](#)

- 1 Diagramme du flux entre les deux composants : humain → prompt → orchestrateur → prompt étendu → modèle → complétion → orchestrateur → réponse → humain. Flux aller (prompt) et retour (réponse) distincts entre chaque composant. Trois nœuds : humain (gris, neutre), orchestrateur (teal), modèle/LLM (violet). Style épuré, flèches directionnelles, labels courts. ↑
- 2 Graphique axes surface (x) / prix (y). Points représentant les transactions. Droite de régression représentant le prix moyen au m². Ligne verticale rouge partant de la surface de l'appartement cible, ligne horizontale arrivant à son prix estimé. Style pédagogique, épuré. ↑
- 3 Mêmes données que immo-regression, avec quelques points aberrants bien exagérés (un bien bradé, un vendu très au-dessus du marché). Montrer comment ils dévient la droite de régression. ↑
- 4 Modèle à deux pentes : une pour les petites surfaces (prix/m² élevé), une pour les grandes (prix/m² plus faible). Illustre l'enrichissement du modèle par rapport à immo-regression. ↑
- 5 Données avec une structure visible (courbe), modèle trop simple (droite) qui ne capture pas la tendance. Le modèle "n'est pas assez intelligent". ↑
- 6 Même données, modèle trop complexe dont la courbe passe par tous les points mais part dans des directions improbables entre les données. Le modèle "apprend par cœur" au lieu de généraliser. ↑
- 7 Distinction visuelle : zone entre les données d'entraînement (interpolation, en vert) et zones en dehors (extrapolation, en rouge). Montrer qu'extrapoler amplifie les erreurs du modèle. ↑
- 8 Schéma de la structure du message envoyé au modèle : [prompt système] + [historique de la conversation] + [dernière question]. Mettre en évidence que l'historique grossit à chaque échange. ↑
- 9 Schéma de la structure complète du prompt étendu avec toutes les couches : [prompt système] → [instructions utilisateur] → [historique de la session] → [dernier prompt]. Variante enrichie de la figure prompt-étendu-historique. ↑
- 10 Illustration du mécanisme de dilution de l'attention : un contexte court où l'attention est concentrée sur les éléments pertinents, vs un contexte long où elle se disperse sur de nombreux éléments peu pertinents. ↑

Table des encadrés ↑

N°	TITRE
1	Explication de la polysémie : pourquoi le langage est intrinsèquement ambigu. ↑